

A Physics-Informed Neural Surrogate for Model Predictive Control of a Single-Track Vehicle: A Controlled Comparison with Exact, Black-Box and Linearised Prediction Models

Ahmed H. Labib^{1,2}, Ahmed W. elMarsafy^{1,2}, Ziad M. elGendy^{1,2},
Zeyad E. Farghaly^{1,2}, Ziad H. Karawia^{1,2}, Zeyad M.
AbdulFattah^{1,2}, and Mohammed D. ElHosary^{1,2}

¹Faculty of Engineering, Cairo University

²Department of Electronics and Communications Engineering

Abstract

Physics-informed neural networks for control (PINC) can serve as fast prediction models in model predictive control (MPC), but it is rarely clear how much the physics term itself contributes. We study this for a single-track vehicle model with coupled longitudinal and lateral dynamics, comparing a PINC surrogate against the same network trained on data only, the exact model and a linearised model, all inside the same MPC. We first show that the usual residual normalisation lets the longitudinal-velocity term dominate the physics loss, so that physics appears to hurt accuracy; scaling the network output per state removes the problem. With this scaling the physics term reduced the error of the learned time derivative 13-fold and the 50-step prediction error 2.2-fold over five seeds, and the gains grew as training data shrank. In closed loop, PINC-MPC tracked four manoeuvres as well as the exact-model controller, while the data-only surrogate raised the speed error by about 50 to 70 % on three of them. The surrogate was not faster than the compiled exact model for horizons up to ten steps and 1.47 times faster at forty.

Keywords: physics-informed neural networks, model predictive control, vehicle dynamics, surrogate models, benchmark

1 Introduction

Road crashes kill more than 1.19 million people a year [World Health Organization, 2023], and automated vehicles are expected to reduce that number. To do so they have to track speed and path references under sensor noise, disturbances and imperfect models [Shiwakoti et al., 2020, Wei et al., 2019]. Model predictive control (MPC) handles the input and state constraints of this problem directly [Rawlings et al., 2017], but most of its computation goes into simulating the prediction model inside the optimiser. Physics-informed neural networks (PINNs) [Raissi et al., 2019], and the control-oriented variant PINC [Antonelo et al., 2024], replace that simulation with a single network evaluation per control period, and they can be trained from a mix of simulated data and the governing equations.

Two recent studies use PINN surrogates inside vehicle MPC [Li and Liu, 2024, Jin et al., 2024] and report better tracking than a conventional MPC. Neither compares against an MPC that uses the exact model with the same solver, or against the same network trained without the physics term. It is therefore hard to say whether the gains come from the physics loss, from the network, or from differences in the controllers being compared.

We address this with a controlled comparison. Our contributions are:

1. a PINC surrogate for a six-state single-track model with coupled longitudinal and lateral dynamics, with an exact initial condition and a per-state output scaling that keeps all four physics residuals balanced;
2. evidence, from derivative errors and long-horizon predictions over five seeds, that the network learns the dynamics rather than only fitting end states, and that this holds from 10^2 to 2×10^4 training trajectories;
3. a benchmark in which the exact model, the PINC network, a data-only network and a linearised model are used inside the same MPC and tested against the same plant on four manoeuvres, 30 seeds each;
4. a loss and architecture study, including the effect of the physics weight λ and of each residual term.

2 Related Work

MPC for automated vehicles. MPC repeatedly solves a finite-horizon optimal control problem and applies the first input [Rawlings et al., 2017]. Minh et al.

[2020] linearise the kinematic bicycle model about a reference and solve a quadratic programme; our linearised baseline follows the same idea with a dynamic model. Min-max MPC [Zhou et al., 2022], sliding-mode control [Young et al., 1999] and H_∞ control [Oudjama et al., 2023] treat model uncertainty in the design. We do not use them here and instead test robustness by perturbing the plant.

Physics-informed neural networks. PINNs add the residual of the governing equations, computed by automatic differentiation at collocation points, to the data loss [Raissi et al., 2019]; Farea et al. [2024] give a recent survey. They have been applied to nonlinear fibre optics [Jiang et al., 2021] and to identifying adaptive cruise control parameters from car-following data [Apostolakis and Ampountolas, 2024]. PINC [Antonelo et al., 2024] adds the initial state and the control input to the network input, so one evaluation predicts the state anywhere in a control period and repeated evaluations give a discrete-time model for MPC. We use this architecture, loss and chaining.

PINN surrogates in vehicle MPC. The closest works are Li and Liu [2024] and Jin et al. [2024]. We differ from them in four ways: we use a dynamic model with lateral velocity instead of a kinematic one, and test against a plant with tyre saturation; we keep the cost, solver and warm start identical across prediction models; we include the exact model and a physics-free network as baselines; and we report confidence intervals and paired tests over seeds.

3 Vehicle Model

We use a single-track model [Rajamani, 2012] with state $x = [v_x, v_y, r, \psi, X, Y]$ (longitudinal and lateral velocity in the body frame, yaw rate, yaw angle and position) and input $u = [F_x, \delta]$ (longitudinal force and front steer angle):

$$m(\dot{v}_x - v_y r) = F_x - \frac{1}{2}\rho C_d A v_x^2 - F_{rr} \tanh(v_x/0.1) - F_{yf} \sin \delta, \quad (1)$$

$$m(\dot{v}_y + v_x r) = F_{yf} \cos \delta + F_{yr}, \quad (2)$$

$$I_z \dot{r} = l_f F_{yf} \cos \delta - l_r F_{yr}, \quad (3)$$

$$\dot{\psi} = r, \quad \dot{X} = v_x \cos \psi - v_y \sin \psi, \quad \dot{Y} = v_x \sin \psi + v_y \cos \psi. \quad (4)$$

The slip angles are $\alpha_f = \delta - \text{atan2}(v_y + l_f r, v_x)$ and $\alpha_r = -\text{atan2}(v_y - l_r r, v_x)$. Tyre forces are linear, $F_y = C_\alpha \alpha$, except in the robustness tests, where we also use the saturating Fiala model with friction limit μF_z [Pacejka, 2012]. The parameters

are $m = 1500$ kg, $I_z = 2500$ kg m², $l_f = l_r = 1.4$ m, $C_{\alpha f} = C_{\alpha r} = 5 \times 10^4$ N/rad, $C_d = 0.3$, $A = 2.2$ m², $\rho = 1.225$ kg/m³ and $F_{rr} = 300$ N.

Linearised about straight driving, (2)–(3) reduce to the usual linear bicycle model, with yaw damping $-(l_f^2 C_{\alpha f} + l_r^2 C_{\alpha r})/(I_z v_x)$. The sign of the rear slip angle matters here: the correct linearisation is $\alpha_r \approx (l_r r - v_y)/v_x$, and with the common sign error $\alpha_r \approx -(v_y + l_r r)/v_x$ the two yaw-damping terms cancel for a symmetric car. We checked the implementation against the linear model (eigenvalues, Jacobian, steady-state yaw gain) and confirmed fourth-order convergence of the fixed-step RK4 integrator. The plant is integrated at 1 ms. Actuator dynamics are neglected and the input is held constant over each control period.

4 PINC Surrogate

4.1 Network

The network predicts $s = [v_x, v_y, r, \psi]$; X and Y are integrated from s outside it. The input is $z = [t/T, s_0/S_x, u/S_u]$ with $T = 0.1$ s, $S_x = [30, 1.5, 0.6, 0.5]$ and $S_u = [6000, 0.3]$. We impose the initial condition exactly and scale the network output per state:

$$\hat{s}(t) = \hat{s}_0 + \frac{t}{T} D \text{NN}(z), \quad D = \text{diag}\left(\frac{S_f T}{S_x}\right), \quad (5)$$

where $\hat{s} = s/S_x$ and S_f is the characteristic rate of each state defined below. The matrix D makes the network output an increment of order one for every state. Without it the network has to resolve a v_x increment that is about a hundred times smaller than its scale, and small slope errors in v_x dominate the physics loss (Section 6.1). NN is a fully connected tanh network with 8 layers of 64 units (29 892 parameters) and Glorot initialisation.

4.2 Loss

Let $f(s, u)$ be the first four rows of (1)–(4). At a collocation point the residual is

$$R = \frac{\dot{s} - f(s, u)}{S_f}, \quad \dot{s} = \frac{S_x}{T} \frac{\partial \hat{s}}{\partial (t/T)}, \quad (6)$$

where the derivative is taken by forward-mode automatic differentiation with respect to the time input and $S_f = [2.035, 9.41, 5.296, 0.3466]$ is the standard devi-

ation of each component of f over the training domain. The loss is

$$\mathcal{L} = \mathcal{L}_{\text{data}} + w_{\text{ic}} \mathcal{L}_{\text{ic}} + \lambda \mathcal{L}_{\text{phys}}, \quad (7)$$

with $\mathcal{L}_{\text{data}}$ the mean squared error against integrated trajectories in scaled units and $\mathcal{L}_{\text{phys}}$ the mean of R^2 . The initial-condition term $\mathcal{L}_{\text{ic}}$ is zero under (5) and is only used when that constraint is relaxed.

4.3 Data and training

Initial states are sampled uniformly from $v_x \in [5, 25]$ m/s, $v_y \in [-1.5, 1.5]$ m/s, $r \in [-0.6, 0.6]$ rad/s and $\psi \in [-0.5, 0.5]$ rad, and inputs from $F_x \in [-6000, 3000]$ N and $\delta \in [-0.3, 0.3]$ rad. Each sample is integrated with RK4 and read out at a random time in $(0, T]$. We use 20 000 training, 4 000 validation and 4 000 test trajectories, plus a separate extrapolation set with $v_x \in [25, 30]$ m/s. 20 000 collocation points are redrawn every epoch. Training runs Adam [Kingma and Ba, 2015] (learning rate 0.001, cosine decay, 300 epochs, batch 1024) followed by 500 L-BFGS iterations [Liu and Nocedal, 1989], and keeps the weights with the lowest validation loss.

5 MPC and Benchmark Design

5.1 Controller

Every controller minimises, over the normalised inputs $\tilde{u}_k = u_k/S_u$,

$$\begin{aligned} J = & \sum_{k=1}^{N-1} e_k^\top Q e_k + e_N^\top P e_N + \sum_{k=0}^{N-1} (\tilde{u}_k^\top R \tilde{u}_k + \Delta \tilde{u}_k^\top R_\Delta \Delta \tilde{u}_k) \\ & + w_r \sum_{k=1}^N \max(0, |r_k| - r_{\max})^2, \end{aligned} \quad (8)$$

where e_k is the tracking error in $[v_x, v_y, r, \psi, X, Y]$ and X, Y are integrated from the predicted states with the trapezoidal rule. We use $Q = P = \text{diag}(1, 0, 0, 10, 0, 1)$, $R = \text{diag}(0.05, 0.05)$, $R_\Delta = \text{diag}(0.5, 0.5)$, $r_{\max} = 0.5$ rad/s, $w_r = 100$ and $N = 10$. The cost and its exact gradient are compiled with XLA in TensorFlow [Abadi et al., 2016] and passed to SLSQP [Kraft, 1988, Virtanen et al., 2020]. Solver settings and the warm start (the previous solution shifted by one step) are the same for every prediction model.

5.2 Prediction models

NMPC-RK4 the exact model with nominal parameters, integrated by RK4 with a 0.01 s step. This is the accuracy ceiling.

PINC-MPC the PINC network evaluated at $t = T$ and chained over the horizon.

Black-box-MPC the same network trained with $\lambda = 0$.

LTV-MPC the exact model linearised along the warm-start trajectory at each solve.

All controllers use nominal parameters; parameter changes and disturbances are applied to the plant only.

5.3 Simulation setup

The plant state is measured with Gaussian noise of standard deviation $[0.05, 0.05, 0.002, 0.005, 0.05, 0.05]$ (in m/s, m/s, rad/s, rad, m and m) on $[v_x, v_y, r, \psi, X, Y]$, typical of an RTK-GNSS/INS receiver [u-blox AG, 2023] and an automotive MEMS gyroscope [Robert Bosch GmbH, 2020]. We use four references: a speed sinusoid $20 + 2 \sin(0.5t)$ m/s, a +3 m/s speed step at 2 s and back at 6 s, a 3.5 m lane change over 40 m at 20 m/s, and the ISO 3888-2 double lane change [International Organization for Standardization, 2011] at 12 m/s. Each seed changes the noise and perturbs the initial speed ($\sigma = 0.5$ m/s). We report RMSE and related error statistics, control effort $\sum_k \tilde{u}_k^\top R \tilde{u}_k T$ and solve time, with 95 % bootstrap confidence intervals over seeds and paired Wilcoxon signed-rank tests against NMPC-RK4.

6 Results

6.1 Model selection

The network size was fixed by an architecture study of 56 configurations run before the output scaling D was introduced (Tables 1 and 2). Depth helped more than width, and none of the regularisers or skip connections improved the fit. We use 8×64 , which was best at $\lambda = 0.01$ in that study and has fifteen times fewer parameters than the best network at $\lambda = 0$.

Table 3 shows the effect of λ and of each loss term with the output scaling in place. Validation error has a clear minimum at $\lambda = 0.01$ (6.1×10^{-7} , against

Table 1: Validation data loss of plain fully-connected networks (depth $\times$ width) for three values of λ ; same data, seeds and optimiser budget for every network.

λ	2×64	4×64	8×64	4×128	8×128	8×256
0	4.7×10^{-6}	2.4×10^{-6}	1.0×10^{-6}	2.1×10^{-6}	5.5×10^{-7}	5.0×10^{-7}
0.01	2.6×10^{-4}	1.5×10^{-4}	5.1×10^{-5}	1.8×10^{-4}	5.2×10^{-5}	1.9×10^{-4}
0.1	1.0×10^{-3}	5.2×10^{-4}	2.7×10^{-4}	4.8×10^{-4}	3.3×10^{-4}	1.8×10^{-3}

Table 2: Validation data loss of regularised and skip-connected variants against the plain network of the same size.

variant	$4 \times 128, \lambda = 0$	$4 \times 128, \lambda = 0.01$	$6 \times 256, \lambda = 0$	$6 \times 256, \lambda = 0.01$
plain	2.1×10^{-6}	1.8×10^{-4}	6.1×10^{-7}	2.1×10^{-4}
dropout 0.05	4.3×10^{-6}	4.6×10^{-4}	1.8×10^{-6}	8.1×10^{-4}
dropout 0.1	9.7×10^{-5}	4.8×10^{-4}	1.8×10^{-4}	1.9×10^{-3}
skip connections	4.4×10^{-6}	2.8×10^{-4}	2.8×10^{-6}	5.1×10^{-4}
ResNet blocks	3.2×10^{-6}	2.7×10^{-4}	3.1×10^{-6}	1.6×10^{-3}
layer normalisation	2.1×10^{-6}	1.9×10^{-4}	1.8×10^{-6}	4.3×10^{-4}

9.2×10^{-7} without physics) and rises again for larger weights, so we use $\lambda = 0.01$ throughout. The exact initial condition is important: relaxing it raised the v_x test error from 4.4×10^{-5} to 3.7×10^{-3} . Removing the v_x residual raised the v_x error to 8.8×10^{-4} , twenty times higher, while the other three residuals mattered less. Without the scaling D the same residual had the opposite effect and degraded the whole model: in the earlier architecture study, adding the physics term at $\lambda = 0.01$ raised the validation loss of the 8×64 network about fifty-fold (Table 1). The per-state scaling is what makes the physics term useful. A larger 6×256 network reached a validation error of 3.0×10^{-7} , half that of the default, at eleven times the parameter count; since the architecture study predates the scaling, the default size may not be optimal.

6.2 Does the network learn the dynamics?

A low one-step error does not show that a network has learned the dynamics; it may fit end states while its time derivative is wrong, and such errors accumulate when predictions are chained over an MPC horizon. We therefore compare the time derivative of each network with the true right-hand side along held-out trajectories, and measure chained predictions over 10 and 50 control periods.

Table 4 shows how these quantities change with λ . The derivative error falls steadily as the physics weight grows, from 0.144 for the data-only network to 0.010 at $\lambda = 0.01$, and long-horizon predictions improve with it. Very large weights still

Table 3: Physics-weight sweep and ablations at the default 8×64 network (seed 0). Validation data loss (scaled MSE on held-out trajectories), validation physics loss, per-state test NRMSE and all-state extrapolation NRMSE. Rows below the λ sweep use $\lambda = 0.01$.

ablation	λ	val. data	val. phys.	test NRMSE $v_x / v_y / r / \psi$	extrap. (all)
$\lambda = 0$	0	9.2×10^{-7}	2.1×10^{-2}	$7.4 \times 10^{-4} / 1.2 \times 10^{-3} / 1.1 \times 10^{-3} / 6.4 \times 10^{-4}$	3.0×10^{-3}
$\lambda = 0.01$	0.01	6.1×10^{-7}	1.2×10^{-4}	$4.4 \times 10^{-5} / 1.1 \times 10^{-3} / 1.1 \times 10^{-3} / 2.8 \times 10^{-4}$	3.1×10^{-3}
$\lambda = 0.1$	0.1	6.9×10^{-7}	4.6×10^{-5}	$2.6 \times 10^{-5} / 1.1 \times 10^{-3} / 1.2 \times 10^{-3} / 2.0 \times 10^{-4}$	3.5×10^{-3}
$\lambda = 1$	1	1.3×10^{-6}	4.6×10^{-5}	$2.3 \times 10^{-5} / 1.4 \times 10^{-3} / 1.7 \times 10^{-3} / 1.9 \times 10^{-4}$	5.7×10^{-3}
$\lambda = 10$	10	1.4×10^{-6}	4.7×10^{-5}	$2.3 \times 10^{-5} / 1.5 \times 10^{-3} / 1.8 \times 10^{-3} / 2.0 \times 10^{-4}$	5.9×10^{-3}
$\lambda = 100$	100	1.3×10^{-6}	4.5×10^{-5}	$2.2 \times 10^{-5} / 1.4 \times 10^{-3} / 1.7 \times 10^{-3} / 1.9 \times 10^{-4}$	5.7×10^{-3}
2k collocation pts	0.01	7.5×10^{-7}	1.6×10^{-4}	$6.0 \times 10^{-5} / 1.2 \times 10^{-3} / 1.2 \times 10^{-3} / 3.1 \times 10^{-4}$	3.9×10^{-3}
100k collocation pts	0.01	6.2×10^{-7}	1.1×10^{-4}	$4.1 \times 10^{-5} / 1.1 \times 10^{-3} / 1.1 \times 10^{-3} / 2.9 \times 10^{-4}$	3.4×10^{-3}
2×64	0.01	2.8×10^{-6}	5.0×10^{-4}	$9.9 \times 10^{-5} / 2.4 \times 10^{-3} / 2.1 \times 10^{-3} / 5.3 \times 10^{-4}$	7.9×10^{-3}
6×256	0.01	3.0×10^{-7}	5.4×10^{-5}	$2.8 \times 10^{-5} / 8.0 \times 10^{-4} / 6.9 \times 10^{-4} / 2.1 \times 10^{-4}$	3.7×10^{-3}
soft IC	0.01	9.8×10^{-5}	5.4×10^{-3}	$3.7 \times 10^{-3} / 1.5 \times 10^{-2} / 1.2 \times 10^{-2} / 4.3 \times 10^{-3}$	2.6×10^{-2}
skip connections	0.01	3.1×10^{-6}	3.8×10^{-4}	$8.1 \times 10^{-5} / 2.4 \times 10^{-3} / 2.4 \times 10^{-3} / 5.7 \times 10^{-4}$	5.9×10^{-3}
ResNet blocks	0.01	2.6×10^{-6}	3.4×10^{-4}	$8.4 \times 10^{-5} / 2.3 \times 10^{-3} / 2.2 \times 10^{-3} / 5.4 \times 10^{-4}$	4.3×10^{-3}
no v_x residual	0.01	7.7×10^{-7}	5.0×10^{-5}	$8.8 \times 10^{-4} / 1.0 \times 10^{-3} / 1.1 \times 10^{-3} / 2.9 \times 10^{-4}$	2.9×10^{-3}
no v_y residual	0.01	6.8×10^{-7}	1.0×10^{-4}	$4.5 \times 10^{-5} / 1.2 \times 10^{-3} / 1.1 \times 10^{-3} / 2.8 \times 10^{-4}$	3.4×10^{-3}
no r residual	0.01	6.4×10^{-7}	1.1×10^{-4}	$4.4 \times 10^{-5} / 1.1 \times 10^{-3} / 1.1 \times 10^{-3} / 2.9 \times 10^{-4}$	3.2×10^{-3}
no ψ residual	0.01	8.1×10^{-7}	9.8×10^{-5}	$4.7 \times 10^{-5} / 1.2 \times 10^{-3} / 1.2 \times 10^{-3} / 7.3 \times 10^{-4}$	3.0×10^{-3}

reduce the derivative error but start to hurt extrapolation, which is consistent with the validation optimum at $\lambda = 0.01$.

Table 4: Effect of the physics weight on what the network learns (one seed). $\dot{s}$ error: RMS difference between the network’s time derivative and the true dynamics along held-out trajectories, in units of S_f . Remaining columns: normalised RMSE of chained predictions.

λ	$\dot{s}$ error	10 steps	50 steps	50 steps (extrap.)
0	0.144	6.6×10^{-3}	2.7×10^{-2}	9.2×10^{-2}
1e-05	0.083	6.2×10^{-3}	2.8×10^{-2}	5.8×10^{-2}
0.0001	0.043	4.9×10^{-3}	2.0×10^{-2}	4.8×10^{-2}
0.001	0.022	5.0×10^{-3}	1.8×10^{-2}	3.0×10^{-2}
0.01	0.010	2.9×10^{-3}	9.2×10^{-3}	3.2×10^{-2}
0.1	0.007	2.8×10^{-3}	8.2×10^{-3}	4.1×10^{-2}
1	0.006	2.8×10^{-3}	8.3×10^{-3}	3.6×10^{-2}

Table 5 repeats the comparison over five seeds, each with a different training set and initialisation. With 2×10^4 training trajectories, PINC was better on every metric and every seed: the derivative error was 13 times lower, the 50-step error 2.2 times lower in the training domain and 2.6 times lower outside it ($p \leq 0.007$). With only 100 trajectories the gap widened to a 14.29-fold reduction in one-step error and a 16.9-fold reduction in 50-step error.

Table 5: PINC ($\lambda = 0.01$) against the same network trained on data only, five seeds per training-set size N . Ratio: geometric mean of data-only / PINC; p : paired t -test on log values.

N	metric	PINC	data-only	ratio	PINC better	p
20 000	one-step test error	7.9×10^{-4}	9.3×10^{-4}	1.2	5/5	0.003
	$\dot{s}$ error	1.0×10^{-2}	1.4×10^{-1}	13.1	5/5	9×10^{-7}
	10-step error	3.4×10^{-3}	6.0×10^{-3}	1.7	5/5	0.007
	50-step error	1.1×10^{-2}	2.5×10^{-2}	2.2	5/5	0.005
	50-step error (extrap.)	3.7×10^{-2}	9.5×10^{-2}	2.6	5/5	0.002
100	one-step test error	2.0×10^{-3}	2.9×10^{-2}	14.3	5/5	4×10^{-7}
	$\dot{s}$ error	1.2×10^{-2}	2.2×10^{-1}	18.2	5/5	5×10^{-7}
	10-step error	5.2×10^{-3}	9.6×10^{-2}	18.1	5/5	1×10^{-6}
	50-step error	1.3×10^{-2}	2.3×10^{-1}	16.9	5/5	4×10^{-6}
	50-step error (extrap.)	6.7×10^{-2}	4.9×10^{-1}	7.2	5/5	6×10^{-4}

6.3 Open-loop accuracy

Table 6 and Figure 1 compare all prediction models over 200 initial states and 20 random input sequences. For both networks the error at $t = 0.25T$ is smaller than at $t = T$, consistent with a continuous-time model rather than a fit to the end of the period only. PINC is 1.4 times more accurate than the data-only network after one step and 3.9 times more accurate after 50 steps (7.7×10^{-3} against 3.0×10^{-2}). Outside the training range the 50-step advantage is 2.7-fold. After one step both networks are about two orders of magnitude more accurate than the linearised model; after 50 steps the margin shrinks to 4–16-fold in the training domain. No chained prediction diverged.

Table 6: Surrogate NRMSE (RMSE / S_x , all states, mean over 200 initial states $\times$ 20 control sequences). One-step at fractional times and chained prediction over 10 and 50 control periods.

region	model	$t = 0.25T$	$t = T$	10 steps	50 steps
in-domain	PINC	4.8×10^{-4}	8.5×10^{-4}	3.1×10^{-3}	7.7×10^{-3}
	Black-box	5.0×10^{-4}	1.2×10^{-3}	7.0×10^{-3}	3.0×10^{-2}
	LTI (20 m/s)	2.4×10^{-2}	7.3×10^{-2}	1.3×10^{-1}	1.2×10^{-1}
	RK4 ($\Delta t = 0.01$ s)	9.7×10^{-8}	6.8×10^{-8}	1.4×10^{-8}	1.4×10^{-8}
extrapolation	PINC	1.2×10^{-3}	2.9×10^{-3}	2.5×10^{-2}	3.0×10^{-2}
	Black-box	9.0×10^{-4}	3.2×10^{-3}	3.2×10^{-2}	8.2×10^{-2}
	LTI (20 m/s)	2.0×10^{-2}	7.5×10^{-2}	3.4×10^{-1}	2.2×10^{-1}
	RK4 ($\Delta t = 0.01$ s)	4.2×10^{-9}	5.3×10^{-9}	5.2×10^{-9}	4.8×10^{-9}

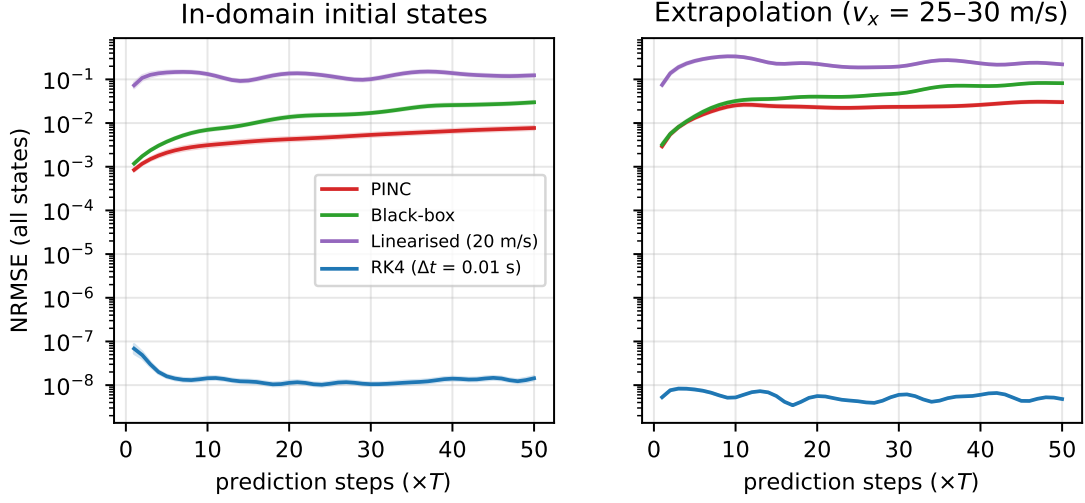


Figure 1: Prediction error against number of chained steps (mean and 95 % CI over initial states).

6.4 Data efficiency

Table 7 and Figure 2 repeat the comparison for training sets of 10^2 to 10^5 trajectories, with five seeds per size and the same optimisation budget. PINC was more accurate at every size, and the gap grew as data became scarce: 14-fold with 10^2 trajectories (2.0×10^{-3} against 2.9×10^{-2}), 1.3-fold with 10^3 and about 1.1-fold with 10^5 . With 10^4 trajectories PINC was already more accurate than the data-only network trained on 10^5 . Most of the gain at large training sets comes from v_x , whose error was about 16 times lower; the other states were close to even. The one-step extrapolation error was slightly higher for PINC at the two largest sizes (3.2×10^{-3} against 3.0×10^{-3} at 10^5), although its chained predictions extrapolate better (Section 6.2).

Table 7: Test NRMSE (all states, in-domain test set) versus number of training trajectories, mean [95% bootstrap CI] over 5 seeds; identical optimiser budget for every run.

arm	$N = 10^2$	$N = 10^3$	$N = 10^4$	$N = 10^5$
PINC ($\lambda = 0.01$)	2.0×10^{-3}	1.1×10^{-3}	8.2×10^{-4}	7.9×10^{-4}
black-box ($\lambda = 0$)	2.9×10^{-2}	1.4×10^{-3}	9.4×10^{-4}	9.0×10^{-4}

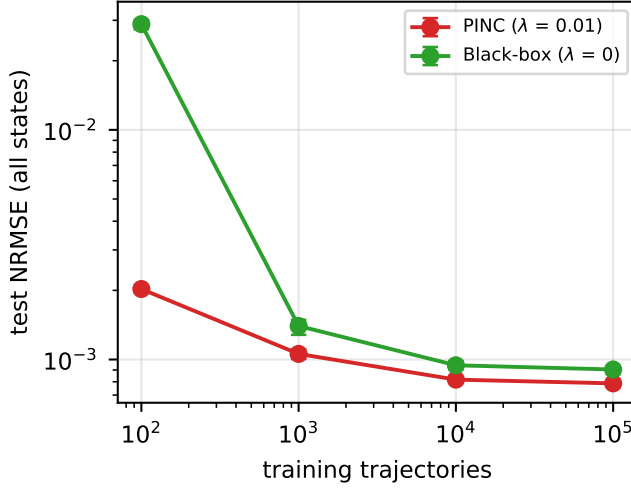


Figure 2: Test error against training-set size (mean and 95 % CI over five seeds).

6.5 Closed-loop tracking

Table 8 summarises the 480 closed-loop runs; the solver converged in all of them. PINC-MPC matched NMPC-RK4 to within 1 % in speed and lateral RMSE on every manoeuvre (its yaw-angle error was up to 7 % higher, on the speed step), and the paired tests found no significant difference in speed tracking on the sinusoid ($p = 0.18$). The data-only controller matched the lateral tracking on the lane changes but was worse on the speed sinusoid on all 30 seeds, with a median speed error 1.84 times that of the exact-model controller ($p = 1.9 \times 10^{-9}$). This matches its larger long-horizon error: speed errors build up slowly and only a surrogate that predicts well over the whole horizon can correct them in time. On the speed step all nonlinear controllers are limited by the 3000 N force bound, and only the linearised controller is clearly worse (0.696 m/s against 0.449). Figure 3 shows one double lane change.

6.6 Solve time

Table 9 and Figure 4 give solve times on a single CPU thread. Once both are compiled, one step of the RK4 model costs 0.085 ms and one network evaluation 0.144 ms, so the network has no inherent speed advantage. PINC-MPC was slightly slower than NMPC-RK4 at $N = 5$ (6.3 against 5.7 ms) and about equal at $N = 10$. It becomes faster from $N \approx 20$, and at $N = 40$ it is 1.47 times faster. We did not measure GPU times.

Table 8: Closed-loop tracking, mean over 30 seeds (measurement noise and initial-state perturbation). Effort is $\sum_k \tilde{u}_k^\top R \tilde{u}_k T$. *: paired Wilcoxon signed-rank test against NMPC-RK4 with $p < 0.05$ (differences of a few percent are statistically resolvable at this seed count; confidence intervals, IAE, maxima and effect sizes for every metric are provided with the code).

reference	arm	RMSE v_x [m/s]	RMSE ψ [rad]	RMSE Y [m]	effort	solve [ms]	success
speed sinusoid	NMPC-RK4	0.058	0.0078	0.024	0.0193	14.5	1.00
	PINC-MPC	0.058	0.0079*	0.024	0.0193*	11.3	1.00
	Black-box-MPC	0.088*	0.0077*	0.026*	0.0201*	11.1	1.00
	LTV-MPC	0.058*	0.0078	0.024	0.0193	28.2	1.00
speed step	NMPC-RK4	0.449	0.0089	0.026	0.0583	14.8	1.00
	PINC-MPC	0.447*	0.0095*	0.026	0.0586*	11.7	1.00
	Black-box-MPC	0.440*	0.0080*	0.028*	0.0588*	11.3	1.00
	LTV-MPC	0.696*	0.0132*	0.052*	0.0942*	29.8	1.00
lane change	NMPC-RK4	0.046	0.0148	0.033	0.0097	14.6	1.00
	PINC-MPC	0.046	0.0149*	0.033*	0.0098*	11.4	1.00
	Black-box-MPC	0.077*	0.0149*	0.034	0.0101*	10.7	1.00
	LTV-MPC	0.046	0.0148	0.033	0.0098*	29.1	1.00
double lane change	NMPC-RK4	0.048	0.0385	0.143	0.0367	17.5	1.00
	PINC-MPC	0.048*	0.0386*	0.144*	0.0368*	13.9	1.00
	Black-box-MPC	0.070*	0.0385*	0.143*	0.0367	13.8	1.00
	LTV-MPC	0.048	0.0385	0.143	0.0367*	31.8	1.00

Table 9: Median / 95th-percentile MPC solve time per control step in ms versus horizon N , CPU single thread, XLA-compiled cost and gradient, SLSQP with identical options and warm start for every arm; last column: one compiled one-step model evaluation (batch 1).

arm	$N = 5$	$N = 10$	$N = 20$	$N = 40$	model call [ms]
NMPC-RK4	5.7 / 7.1	10.6 / 12.1	20.3 / 34.5	47.5 / 55.5	0.085
PINC-MPC	6.3 / 7.6	10.0 / 11.8	14.4 / 18.3	32.4 / 34.3	0.144
Black-box-MPC	6.1 / 7.6	9.6 / 10.5	16.0 / 17.5	29.6 / 30.7	0.139
LTV-MPC	15.7 / 21.0	20.1 / 23.8	24.6 / 32.7	41.3 / 46.5	—

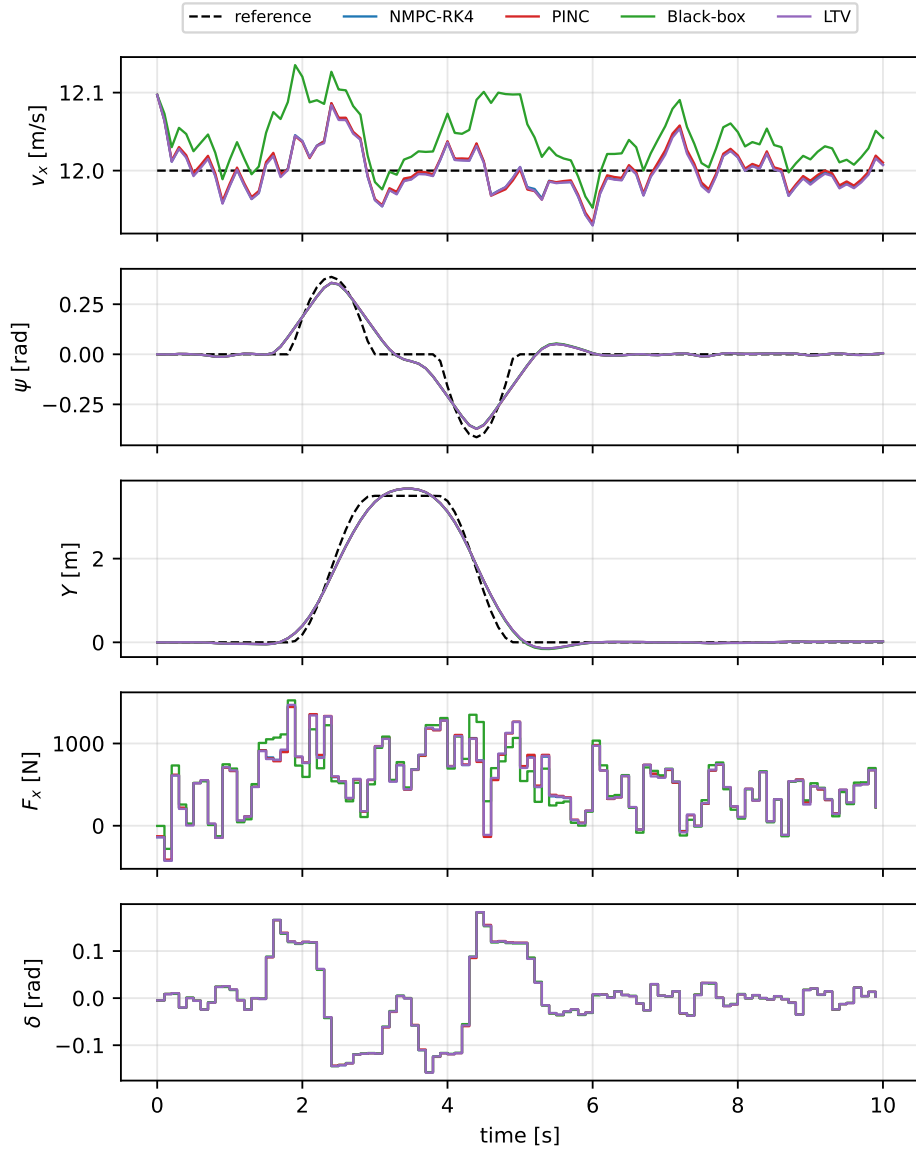


Figure 3: Double lane change at 12 m/s (one noisy run).

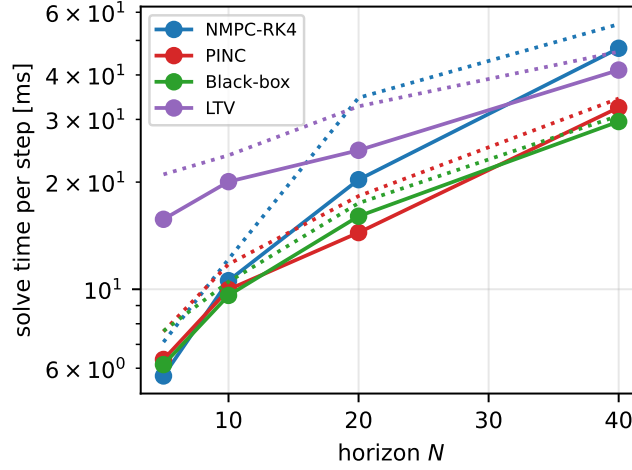


Figure 4: Solve time per control step against horizon length (solid: median, dotted: 95th percentile).

6.7 Robustness to mismatch

Table 10 and Figure 5 show the lane change when the plant differs from the model used by the controllers. All controllers degrade together in lateral tracking; for NMPC-RK4 the lateral RMSE grows from 0.033 m to 0.062 m with 30 % lower cornering stiffness and to 0.075 m at $\mu = 0.4$, and PINC-MPC stays within a few percent of it. In speed tracking the data-only controller carries a constant penalty across all settings (0.077 against 0.046 m/s nominal). The one exception is a -1 kN step disturbance, where its speed error was lower (0.084 against 0.120 m/s): its model bias happens to point in the direction of this disturbance and partly offsets it. We do not read this as a genuine robustness advantage.

Table 10: Robustness to plant-model mismatch on the single lane change (30 seeds). Controllers keep the nominal parameters; only the plant is perturbed. Columns 2–5: RMSE Y [m]; columns 6–9: RMSE v_x [m/s]. All solves succeeded in every setting.

plant setting	NMPC-RK4	PINC	Black-box	LTV	NMPC-RK4	PINC	Black-box	LTV
nominal	0.0334	0.0332	0.0337	0.0334	0.046	0.046	0.077	0.046
m-20%	0.0364	0.0371	0.0367	0.0364	0.044	0.044	0.076	0.044
m+20%	0.0424	0.0414	0.0426	0.0424	0.049	0.049	0.079	0.049
Ca-30%	0.0622	0.0608	0.0622	0.0622	0.046	0.047	0.079	0.046
Ca+30%	0.0384	0.0391	0.0386	0.0384	0.046	0.046	0.076	0.046
fiala $\mu = 1.0$	0.0378	0.0370	0.0380	0.0378	0.046	0.046	0.077	0.046
fiala $\mu = 0.7$	0.0426	0.0415	0.0428	0.0426	0.046	0.046	0.078	0.046
fiala $\mu = 0.4$	0.0745	0.0733	0.0752	0.0746	0.048	0.048	0.081	0.048
Fx step -1000 N at 4 s	0.0334	0.0332	0.0338	0.0334	0.120	0.122	0.084	0.122

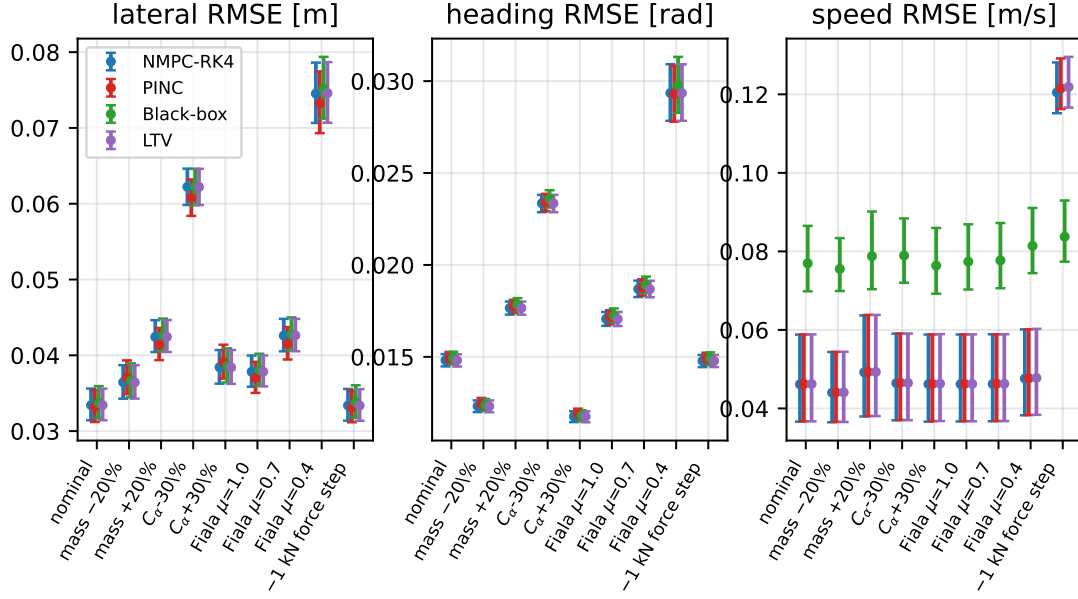


Figure 5: Tracking error of the lane change for each plant perturbation (mean and 95 % CI over 30 runs).

7 Limitations

All results are from simulation, and the plant belongs to the same model family as the controllers' prediction model; hardware-in-the-loop and vehicle tests are left for future work. We give no stability or recursive feasibility guarantees, and the yaw-rate limit is a soft penalty. The surrogate was trained with linear tyres only, and actuator dynamics are neglected. The network size was chosen before the output scaling was introduced, and the ablations suggest a larger network would be more accurate at a higher computational cost. Solve times depend on the processor and compiler, and a GPU or a different solver would move the crossover horizon.

8 Conclusion

Whether the physics term helps a PINC surrogate depends strongly on how its residual is scaled. With the usual normalisation the longitudinal-velocity residual dominated the loss and physics appeared to hurt accuracy; a per-state output scaling removed this. With it, the physics term made the network learn the dynamics: its time derivative was an order of magnitude more accurate than that of the same network trained on data alone, its long-horizon predictions two to

seventeen times more accurate depending on the amount of data, and in closed loop it matched the exact model where the data-only surrogate lost speed accuracy. The surrogate was not faster than a compiled exact model at short horizons, so its practical value lies in cases where the exact model is expensive or unavailable. Experiments on a real vehicle are the next step.

Data and code availability

Code, configuration files and all results are available at <https://github.com/Euro-max/PINNS>. The tables and figures in this paper are generated from the stored results by a script in the repository. Experiments were run on an Intel Core Ultra 9 275HX (6 virtual CPUs) with TensorFlow 2.22.0-rc0.

References

- World Health Organization. Road traffic injuries. <https://www.who.int/news-room/fact-sheets/detail/road-traffic-injuries>, 2023.
- Nirajan Shiwakoti, Peter Stasinopoulos, and Francesco Fedele. Investigating the state of connected and autonomous vehicles: a literature review. *Transportation Research Procedia*, 48:870–882, 2020. doi: 10.1016/j.trpro.2020.08.101.
- Shouyang Wei, Yuan Zou, Xudong Zhang, Tao Zhang, and Xiaoliang Li. An integrated longitudinal and lateral vehicle following control system with radar and vehicle-to-vehicle communication. *IEEE Transactions on Vehicular Technology*, 68(2):1116–1127, 2019. doi: 10.1109/TVT.2018.2890418.
- James B. Rawlings, David Q. Mayne, and Moritz M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, Santa Barbara, CA, 2nd edition, 2017.
- M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378: 686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.
- Eric Aislan Antonelo, Eduardo Camponogara, Laio Oriel Seman, Jean Panaioti Jordanou, Eduardo Rehbein de Souza, and Jomi Fred Hübner. Physics-informed

- neural nets for control of dynamical systems. *Neurocomputing*, 579:127419, 2024. doi: 10.1016/j.neucom.2024.127419. arXiv:2104.02556.
- Y. Li and L. Liu. Physics-informed neural network-based nonlinear model predictive control for automated guided vehicle trajectory tracking. *World Electric Vehicle Journal*, 15(10):460, 2024. doi: 10.3390/wevj15100460.
- L. Jin, L. Liu, X. Wang, M. Shang, and F.-Y. Wang. Physical-informed neural network for MPC-based trajectory tracking of vehicles with noise considered. *IEEE Transactions on Intelligent Vehicles*, 9(1):1305–1319, 2024.
- Vu Trieu Minh, Reza Moezi, Klodian Dhoska, and John Pumwa. Model predictive control for autonomous vehicle tracking. *Journal of Control Engineering*, 56: 560–575, 2020.
- J. Zhou, D. Tian, Z. Sheng, X. Duan, G. Qu, D. Zhao, D. Cao, and X. Shen. Robust min-max model predictive vehicle platooning with causal disturbance feedback. *IEEE Transactions on Intelligent Transportation Systems*, 2022. doi: 10.1109/TITS.2022.3146149.
- K. D. Young, V. I. Utkin, and U. Ozguner. A control engineer’s guide to sliding mode control. *IEEE Transactions on Control Systems Technology*, 7(3):328–342, 1999. doi: 10.1109/87.761053.
- F. Oudjama, A. Boumediene, K. Saidi, and D. Boubekur. Robust speed control in nonlinear electric vehicles using H-infinity control and the LMI approach. *Journal of Intelligent Systems and Control*, 2(3):170–182, 2023. doi: 10.56578/jisc020305.
- Amer Farea, Olli Yli-Harja, and Frank Emmert-Streib. Understanding physics-informed neural networks: Techniques, applications, trends, and challenges. *AI*, 5(3):1534–1557, 2024. doi: 10.3390/ai5030074.
- Xiaotian Jiang, Danshi Wang, Qirui Fan, Min Zhang, Chao Lu, and Alan Pak Tao Lau. Physics-informed neural network for nonlinear dynamics in fiber optics. *Laser & Photonics Reviews*, 2021. doi: 10.1002/lpor.202100483.
- Theocharis Apostolakis and Konstantinos Ampountolas. Physics-inspired neural networks for parameter learning of adaptive cruise control systems. *IEEE Transactions on Vehicular Technology*, 73(10):14291–14301, 2024. doi: 10.1109/TVT.2024.3399918.

- Rajesh Rajamani. *Vehicle Dynamics and Control*. Springer, 2nd edition, 2012. doi: 10.1007/978-1-4614-1433-9.
- Hans B. Pacejka. *Tire and Vehicle Dynamics*. Butterworth-Heinemann, 3rd edition, 2012.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- Dong C. Liu and Jorge Nocedal. On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, 45:503–528, 1989. doi: 10.1007/BF01589116.
- Martín Abadi et al. TensorFlow: Large-scale machine learning on heterogeneous systems, 2016. arXiv:1603.04467.
- Dieter Kraft. A software package for sequential quadratic programming. Technical Report DFVLR-FB 88-28, DLR German Aerospace Center, Institute for Flight Mechanics, 1988.
- Pauli Virtanen et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- u-blox AG. ZED-F9P module data sheet. <https://www.u-blox.com/en/product/zed-f9p-module>, 2023.
- Robert Bosch GmbH. SMI230 inertial measurement unit data sheet. <https://www.bosch-semiconductors.com/>, 2020.
- International Organization for Standardization. ISO 3888-2:2011 passenger cars – test track for a severe lane-change manoeuvre – part 2: Obstacle avoidance. Technical report, ISO, Geneva, 2011.